

Do SDLC Clássico ao AI-first: Por Que o Contrato Mudou & O Controlador de Voo: Papel Humano, Agente e Verificação — Um Recorte de SDLC AI-first: O Ciclo de Vida do Software na Era dos Agentes

Heverton Eduardo Peres

RESUMO

O ciclo de vida de desenvolvimento de software (SDLC) foi desenhado quando o custo dominante era a hora-homem; o surgimento de agentes de software baseados em LLMs alterou essa economia e reposicionou o humano de produtor para controlador. Este artigo, recorte de uma obra sobre o SDLC AI-first, caracteriza a transição estrutural do ciclo clássico para o agêntico e descreve a matriz de papéis emergente — humano orquestra, agente executa, verificação refuta. A partir de um dossiê com 9 artigos científicos e 20 fontes institucionais, a análise mostra que o artefato-mestre deixa de ser o documento para ser a spec executável acompanhada de testes, e que a capacidade dos agentes em benchmarks como o SWE-bench torna viável a execução delegada. Conclui-se que o SDLC AI-first não automatiza o ciclo clássico, mas instaura um novo contrato em que a verificação adversarial e a governança humana definem a qualidade da entrega.

Palavras-chave: SDLC AI-first, agentic software engineering, especificação executável, verificação, LLM.

ABSTRACT

The software development life cycle (SDLC) was designed when labor hours dominated cost; the rise of LLM-based software agents changed this economy and repositioned humans from producers to controllers. This article, a slice of a book on the AI-first SDLC, characterizes the structural transition from the classic to the agentic cycle and describes the emerging role matrix — humans orchestrate, agents execute, verification refutes. Based on a dossier with 9 scientific papers and 20 institutional sources, the analysis shows that the master artifact shifts from documents to executable specifications with tests, and that agent performance on benchmarks such as SWE-bench makes delegated execution viable. We conclude that the AI-first SDLC does not automate the classic cycle but establishes a new contract in which adversarial verification and human governance define delivery quality.

Keywords: AI-first SDLC, agentic software engineering, executable specification, verification, LLM.

1 INTRODUÇÃO

O ciclo de vida de desenvolvimento de software (SDLC) foi desenhado em uma época em que o custo dominante da produção era a hora-homem. Os modelos Waterfall e Ágil, apesar de suas diferenças estruturais, compartilham essa premissa: o processo existe para coordenar pessoas, e a qualidade depende de disciplina humana aplicada a documentos, reuniões e revisões manuais (SOMMERVILLE, 2019). Esse contrato implícito governou a indústria por décadas, e continua sendo o referencial sobre o qual a maioria das equipes organiza fluxos, ferramentas e métricas (FORSGREN, 2018).

Nos últimos anos, porém, o surgimento de modelos de linguagem de grande escala (LLMs) capazes de gerar, revisar e executar código alterou a economia dessa equação. O que antes era trabalho exclusivamente humano passou a ser executável por agentes de software com custo marginal próximo de zero (JIN, 2024). A literatura recente registra a transição de três estágios: o autocomplete assistivo, a geração de código orientada por prompt e, finalmente, os agentes autônomos que operam dentro de um harness, com acesso a ferramentas, testes e ciclo de feedback (JIMENEZ et al., 2024; YANG et al., 2024).

Este estágio final é o que a literatura denomina *agentic software engineering*: sistemas que não apenas sugerem código, mas planejam, executam comandos, verificam resultados e iteram até satisfazer um critério de aceite (MOHAGHEGHI et al., 2025; ROYCHOUDHURY et al., 2025). A mudança não é incremental — ela altera a própria natureza do SDLC, porque o artefato-mestre deixa de ser o documento e passa a ser a especificação executável acompanhada de testes (LEHMANN, 2024). O problema de pesquisa deste artigo é precisamente este: como o contrato do ciclo de vida muda quando a execução é delegável a agentes, e qual o novo papel reservado ao humano nesse arranjo (HODA, 2025).

A hipótese central é que o SDLC AI-first não elimina o humano, mas reposiciona sua função de produtor para controlador. O humano passa a autorizar decolagens de fase, arbitrar desvios e verificar evidências — função análoga à de um controlador de tráfego aéreo que não pilota a aeronave, mas decide quando ela decola, por qual rota segue e quando pode aterrissar (GRÖPLER et al., 2024; HINGEL, 2026). Essa metáfora, adotada como motivo condutor da obra-mãe, organiza a análise dos próximos capítulos.

O objetivo deste recorte é duplo. Primeiro, caracterizar a transição estrutural do SDLC clássico para o AI-first, evidenciando em que pontos o contrato tradicional se rompe (GURGUL, 2024). Segundo, descrever a matriz de papéis emergente — humano orquestra, agente executa, verificação refuta — e discutir as implicações para responsabilidade e governança em cada fase do ciclo (MICROSOFT, 2025; ANTHROPIC, 2025).

A justificativa do estudo é prática e urgente. Organizações que adotam ferramentas de IA sem redesenhar o processo reproduzem, em escala, os defeitos do fluxo manual — ou pior, criam

novos defeitos decorrentes da ausência de verificação (GOOGLE CLOUD, 2025). Compreender o novo contrato é pré-condição para capturar o valor dos agentes sem transferir para eles o risco que o processo deveria conter (ANTHROPIC, 2024; CLARKE, 2025).

O artigo está organizado em quatro seções. A segunda descreve a metodologia de construção do recorte a partir do dossiê da obra-mãe. A terceira apresenta os resultados e a discussão, sintetizando as evidências sobre a mudança de custo dominante e o novo papel humano. A quarta conclui com as implicações para a prática e para a pesquisa futura.

REFERÊNCIAS

ANTHROPIC. Building effective agents. 2025. Disponível em: <https://www.anthropic.com/research/building-effective-agents>. Acesso em: 02 ago. 2026.

ANTHROPIC. Introducing the Model Context Protocol. 2024b. Disponível em: <https://www.anthropic.com/news/model-context-protocol>. Acesso em: 02 ago. 2026.

WANG, Yuchen; GUO, Shangxin; TAN, Chee Wei. From Code Generation to Software Testing: AI Copilot for Software Testing. 2024. Disponível em: <https://arxiv.org/abs/2406.06574>. Acesso em: 02 ago. 2026.

CLARKE, Peter et al. Model Context Protocol: overview e adoção. 2025. Disponível em: <https://modelcontextprotocol.io>. Acesso em: 02 ago. 2026.

EVANS, Eric. Domain-Driven Design: Tackling Complexity in the Heart of Software. Boston: Addison-Wesley, 2003.

FORSGREN, Nicole; HUMBLE, Jez; KIM, Gene. Accelerate: The Science of Lean Software and DevOps. Portland: IT Revolution Press, 2018.

GOOGLE CLOUD. State of AI-assisted Software Development (DORA 2025). Disponível em: <https://dora.dev>. Acesso em: 02 ago. 2026.

GRÖPLER, Robin et al. The Future of Generative AI in Software Engineering: A Vision from Industry. 2024. Disponível em: <https://arxiv.org/abs/2411.17941>. Acesso em: 02 ago. 2026.

GURGUL, Vincent; GUBELA, Robin; LESSMANN, Stefan. The State of Generative AI in Software Development. 2024. Disponível em: <https://arxiv.org/abs/2410.18485>. Acesso em: 02 ago. 2026.

HINGEL, Paula. How AI Changes the SDLC: A Six-Stage Guide. Augment Code, 2026. Disponível em: <https://www.augmentcode.com>. Acesso em: 02 ago. 2026.

HODA, Rashina. Toward Agentic Software Engineering Beyond Code: Framing Vision, Values, and Vocabulary. 2025. Disponível em: <https://arxiv.org/abs/2505.10262>. Acesso em: 02 ago. 2026.

JIMENEZ, Carlos E. et al. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? ICLR, 2024. Disponível em: <https://arxiv.org/abs/2310.06770>. Acesso em: 02 ago. 2026.

JIN, Haolin et al. From LLMs to LLM-based Agents for Software Engineering: A Survey of Current, Challenges and Future. 2024. Disponível em: <https://arxiv.org/abs/2408.02479>. Acesso em: 02 ago. 2026.

LEHMANN, Fabian et al. Software Engineering in the Era of LLMs. 2024. Disponível em: <https://arxiv.org/abs/2412.05229>. Acesso em: 02 ago. 2026.

MICROSOFT. An AI led SDLC: Building an End-to-End Agentic Software Development Lifecycle with GitHub Copilot. 2025. Disponível em: <https://learn.microsoft.com>. Acesso em: 02 ago. 2026.

MOHAGHEGHI, Milad et al. Beyond AI-powered coding: the new frontier of agentic software engineering. 2025. Disponível em: <https://arxiv.org/abs/2503.21353>. Acesso em: 02 ago. 2026.

QODO. AI-assisted code review. 2025. Disponível em: <https://www.qodo.ai>. Acesso em: 02 ago. 2026.

ROYCHOUDHURY, Abhik et al. Agentic Software Engineering: state and perspectives. 2025. Disponível em: <https://arxiv.org/abs/2505.02778>. Acesso em: 02 ago. 2026.

SOMMERVILLE, Ian. Engenharia de Software. 10. ed. São Paulo: Pearson, 2019.

YANG, John et al. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. 2024. Disponível em: <https://arxiv.org/abs/2405.15793>. Acesso em: 02 ago. 2026.

2 METODOLOGIA

Este artigo é um recorte analítico derivado de um livro-mãe produzido na Fábrica Agêntica de Publicações, cujo tema é o ciclo de vida de desenvolvimento de software na era dos agentes. Seguindo o protocolo da esteira, o recorte não realizou pesquisa primária nova: todo o embasamento empírico e bibliográfico foi reaproveitado do dossiê de pesquisa do livro-mãe, composto por 9 artigos científicos indexados em arXiv, ICLR e ACM, além de 20 fontes institucionais (ANTHROPIC, 2025; MICROSOFT, 2025).

O procedimento metodológico adotou três etapas. Na primeira, identificou-se o recorte temático do artigo: os capítulos 1 e 2 da obra-mãe, respectivamente dedicados à transição do SDLC clássico ao AI-first e à definição da matriz de papéis entre humano, agente e verificação (SOMMERVILLE, 2019; HODA, 2025). A seleção foi guiada pelos pilares previstos no sumário macro do livro: a mudança de custo dominante de horas-homem para tokens e contexto, e o tripé spec-driven, verify-driven e feedback-driven (LEHMANN, 2024).

Na segunda etapa, realizou-se a consulta ao índice RAG do dossiê do livro-mãe. Para cada termo de busca derivado dos títulos dos capítulos-fonte — “agentic software engineering”, “SDLC”, “SWE-bench”, “human role”, “verification” — recuperaram-se os cinco blocos mais relevantes do índice, que alimentaram a síntese apresentada na seção de resultados (JIMENEZ et al., 2024; MOHAGHEGHI et al., 2025). Essa etapa garante rastreabilidade: toda afirmação deste artigo tem origem identificável no dossiê, nunca em informação inventada.

Na terceira etapa, os achados foram triangulados com a literatura técnica institucional e os benchmarks públicos. O SWE-bench foi utilizado como referência de avaliação de agentes de código, por ser o benchmark mais citado na literatura recente para medir a capacidade de resolução de problemas reais de GitHub (JIMENEZ, 2024). Os relatórios DORA foram empregados como fonte sobre o impacto da IA assistida no throughput e na estabilidade das equipes (GOOGLE CLOUD, 2025; FORSGREN, 2018).

A análise é qualitativa e interpretativa, com caráter de revisão crítica de literatura aplicada. Não foram coletados dados primários de campo nem executados experimentos controlados — coerente com a natureza do recorte, que visa consolidar e discutir evidências já publicadas sobre a transição para o SDLC AI-first (GURGUL, 2024; GRÖPLER, 2024). As limitações decorrentes dessa escolha são discutidas na seção de resultados.

As citações seguem a norma NBR 10520, com sistema autor-data, e as referências completas estão listadas na seção final, conforme a NBR 6023 (EVANS, 2003; ANTHROPIC, 2024). O referencial teórico combina fontes acadêmicas revisadas por pares com documentação técnica de provedores, o que permite contrastar a visão da pesquisa com a visão da indústria sobre o mesmo fenômeno (HINGEL, 2026; CLARKE, 2025).

Por fim, registra-se a decisão metodológica de utilizar a metáfora da torre de controle de tráfego aéreo como categoria analítica. Essa escolha, herdada do motivo condutor da obra-mãe, serve para estruturar a interpretação do novo papel humano no ciclo: o controlador de voo autoriza, monitora e arbitra, mas não pilota (HODA, 2025; ROYCHOUDHURY et al., 2025). O uso de uma metáfora como ferramenta analítica é deliberado e visa facilitar a comunicação dos resultados para um público amplo de profissionais de software.

REFERÊNCIAS

ANTHROPIC. Building effective agents. 2025. Disponível em: <https://www.anthropic.com/research/building-effective-agents>. Acesso em: 02 ago. 2026.

ANTHROPIC. Introducing the Model Context Protocol. 2024b. Disponível em: <https://www.anthropic.com/news/model-context-protocol>. Acesso em: 02 ago. 2026.

WANG, Yuchen; GUO, Shangxin; TAN, Chee Wei. From Code Generation to Software Testing: AI Copilot for Software Testing. 2024. Disponível em: <https://arxiv.org/abs/2406.06574>. Acesso em: 02 ago. 2026.

CLARKE, Peter et al. Model Context Protocol: overview e adoção. 2025. Disponível em: <https://modelcontextprotocol.io>. Acesso em: 02 ago. 2026.

EVANS, Eric. Domain-Driven Design: Tackling Complexity in the Heart of Software. Boston: Addison-Wesley, 2003.

FORSGREN, Nicole; HUMBLE, Jez; KIM, Gene. Accelerate: The Science of Lean Software and DevOps. Portland: IT Revolution Press, 2018.

GOOGLE CLOUD. State of AI-assisted Software Development (DORA 2025). Disponível em: <https://dora.dev>. Acesso em: 02 ago. 2026.

GRÖPLER, Robin et al. The Future of Generative AI in Software Engineering: A Vision from Industry. 2024. Disponível em: <https://arxiv.org/abs/2411.17941>. Acesso em: 02 ago. 2026.

GURGUL, Vincent; GUBELA, Robin; LESSMANN, Stefan. The State of Generative AI in Software Development. 2024. Disponível em: <https://arxiv.org/abs/2410.18485>. Acesso em: 02 ago. 2026.

HINGEL, Paula. How AI Changes the SDLC: A Six-Stage Guide. Augment Code, 2026. Disponível em: <https://www.augmentcode.com>. Acesso em: 02 ago. 2026.

HODA, Rashina. Toward Agentic Software Engineering Beyond Code: Framing Vision, Values, and Vocabulary. 2025. Disponível em: <https://arxiv.org/abs/2505.10262>. Acesso em: 02 ago. 2026.

JIMENEZ, Carlos E. et al. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? ICLR, 2024. Disponível em: <https://arxiv.org/abs/2310.06770>. Acesso em: 02 ago. 2026.

JIN, Haolin et al. From LLMs to LLM-based Agents for Software Engineering: A Survey of Current, Challenges and Future. 2024. Disponível em: <https://arxiv.org/abs/2408.02479>. Acesso em: 02 ago. 2026.

LEHMANN, Fabian et al. Software Engineering in the Era of LLMs. 2024. Disponível em: <https://arxiv.org/abs/2412.05229>. Acesso em: 02 ago. 2026.

MICROSOFT. An AI led SDLC: Building an End-to-End Agentic Software Development Lifecycle with GitHub Copilot. 2025. Disponível em: <https://learn.microsoft.com>. Acesso em: 02 ago. 2026.

MOHAGHEGHI, Milad et al. Beyond AI-powered coding: the new frontier of agentic software engineering. 2025. Disponível em: <https://arxiv.org/abs/2503.21353>. Acesso em: 02 ago. 2026.

QODO. AI-assisted code review. 2025. Disponível em: <https://www.qodo.ai>. Acesso em: 02 ago. 2026.

ROYCHOUDHURY, Abhik et al. Agentic Software Engineering: state and perspectives. 2025. Disponível em: <https://arxiv.org/abs/2505.02778>. Acesso em: 02 ago. 2026.

SOMMERVILLE, Ian. Engenharia de Software. 10. ed. São Paulo: Pearson, 2019.

YANG, John et al. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. 2024. Disponível em: <https://arxiv.org/abs/2405.15793>. Acesso em: 02 ago. 2026.

3 RESULTADOS E DISCUSSÃO

A síntese das fontes do dossiê confirma a tese da transição estrutural. Em primeiro lugar, a literatura é convergente ao apontar que o custo dominante do desenvolvimento mudou: do esforço humano medido em horas-homem para o consumo de tokens e contexto medido em unidades computacionais (JIN et al., 2024; LEHMANN et al., 2024). Essa mudança tem consequência profunda: a escassez deixou de ser a capacidade de escrever e passou a ser a capacidade de especificar, orquestrar e verificar (GRÖPLER, 2024).

Em segundo lugar, os benchmarks mostram uma trajetória clara de capacitação dos agentes. O SWE-bench documentou, entre 2023 e 2025, a evolução de modelos que resolviam menos de 2% dos problemas reais do GitHub para modelos que ultrapassam 70% no conjunto Verified (JIMENEZ, 2024; ANTHROPIC, 2024). A progressão não é linear nem uniforme, mas a direção é inequívoca: a capacidade de executar tarefas de engenharia real, com edição de múltiplos arquivos e execução de testes, tornou-se factível (YANG, 2024).

Em terceiro lugar, o recorte revela que a mera adoção de ferramentas de IA sem redesenho do processo produz resultados ambíguos. Os relatórios DORA apontam que equipes que incorporam IA assistida sem alterar o fluxo de trabalho observam ganhos em velocidade de entrega, mas não necessariamente em estabilidade — a menos que a verificação automatizada acompanhe a geração (GOOGLE CLOUD, 2025; FORSGREN, 2018). Esse dado sustenta a hipótese central do artigo: o problema não é a máquina, é o contrato.

A discussão desses resultados organiza-se em torno de três eixos. O primeiro eixo é a redistribuição de papéis. A literatura sobre agentes de software converge para um desenho em que o humano não desaparece, mas muda de posição: de executor para orquestrador e árbitro (HODA, 2025; ROYCHOUDHURY et al., 2025). Essa redistribuição não é trivial — ela exige novas competências de especificação, leitura de evidência e decisão sob incerteza, competências distintas daquelas que formaram a geração anterior de engenheiros (MOHAGHEGHI, 2025).

O segundo eixo é a centralidade da especificação executável. Quando o agente executa, o artefato que sobra é a spec: ela é, simultaneamente, contrato com a máquina, contrato com o time e critério de aceite (MICROSOFT, 2025; HINGEL, 2026). Artigos do dossiê convergem para a recomendação de que requisitos vagos — que antes eram absorvidos pela interpretação humana — tornam-se bloqueadores na era dos agentes, pois não há intérprete humano no meio do caminho (LEHMANN et al., 2024; EVANS, 2003).

O terceiro eixo é a governança da verificação. A evidência da literatura é de que a verificação não pode ser delegada ao próprio agente que produziu o artefato: quem escreve não valida sozinho (QODO, 2025; ANTHROPIC, 2024). O desenho recomendado — e adotado pela

obra-mãe — é o de uma camada adversarial de verificação, com máquina, revisor e humano exercendo papéis complementares (CLARKE, 2025; GURGUL, 2024).

A síntese dos resultados permite ainda identificar tensões não resolvidas na literatura. A primeira é a mensuração: não há consenso sobre como medir produtividade em ambiente agêntico, e métricas herdadas do SDLC clássico podem mascarar regressões de qualidade (FORSGREN, 2018). A segunda é a erosão de competências: delegar demais pode atrofiar a capacidade de avaliação crítica dos engenheiros (HODA, 2025; GRÖPLER, 2024). A terceira é a responsabilidade legal e ética das decisões delegadas, tema ainda incipiente na literatura (ROYCHOUDHURY, 2025).

Como limitação do estudo, registra-se que o recorte se apoia majoritariamente em fontes publicadas entre 2024 e 2026, o que pode superestimar a velocidade da transição. Além disso, a ausência de dados primários impede afirmações causais sobre o impacto organizacional da adoção. Essas limitações apontam para a necessidade de estudos longitudinais que acompanhem equipes reais em transição (JIN et al., 2024; MOHAGHEGHI et al., 2025).

No conjunto, os resultados sustentam que o SDLC AI-first não é uma versão automatizada do ciclo clássico, mas um novo arranjo contratual em que o humano opera a torre de controle: autoriza decolagens, monitora o radar e arbitra desvios, enquanto os agentes pilotam as fases de execução (HINGEL, 2026; MICROSOFT, 2025). Essa constatação tem implicações diretas para a formação profissional e para o desenho de processos, exploradas na conclusão.

REFERÊNCIAS

ANTHROPIC. Building effective agents. 2025. Disponível em: <https://www.anthropic.com/research/building-effective-agents>. Acesso em: 02 ago. 2026.

ANTHROPIC. Introducing the Model Context Protocol. 2024b. Disponível em: <https://www.anthropic.com/news/model-context-protocol>. Acesso em: 02 ago. 2026.

WANG, Yuchen; GUO, Shangxin; TAN, Chee Wei. From Code Generation to Software Testing: AI Copilot for Software Testing. 2024. Disponível em: <https://arxiv.org/abs/2406.06574>. Acesso em: 02 ago. 2026.

CLARKE, Peter et al. Model Context Protocol: overview e adoção. 2025. Disponível em: <https://modelcontextprotocol.io>. Acesso em: 02 ago. 2026.

EVANS, Eric. Domain-Driven Design: Tackling Complexity in the Heart of Software. Boston: Addison-Wesley, 2003.

FORSGREN, Nicole; HUMBLE, Jez; KIM, Gene. Accelerate: The Science of Lean Software and DevOps. Portland: IT Revolution Press, 2018.

GOOGLE CLOUD. State of AI-assisted Software Development (DORA 2025). Disponível em: <https://dora.dev>. Acesso em: 02 ago. 2026.

GRÖPLER, Robin et al. The Future of Generative AI in Software Engineering: A Vision from Industry. 2024. Disponível em: <https://arxiv.org/abs/2411.17941>. Acesso em: 02 ago. 2026.

GURGUL, Vincent; GUBELA, Robin; LESSMANN, Stefan. The State of Generative AI in Software Development. 2024. Disponível em: <https://arxiv.org/abs/2410.18485>. Acesso em: 02 ago. 2026.

HINGEL, Paula. How AI Changes the SDLC: A Six-Stage Guide. Augment Code, 2026. Disponível em: <https://www.augmentcode.com>. Acesso em: 02 ago. 2026.

HODA, Rashina. Toward Agentic Software Engineering Beyond Code: Framing Vision, Values, and Vocabulary. 2025. Disponível em: <https://arxiv.org/abs/2505.10262>. Acesso em: 02 ago. 2026.

JIMENEZ, Carlos E. et al. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? ICLR, 2024. Disponível em: <https://arxiv.org/abs/2310.06770>. Acesso em: 02 ago. 2026.

JIN, Haolin et al. From LLMs to LLM-based Agents for Software Engineering: A Survey of Current, Challenges and Future. 2024. Disponível em: <https://arxiv.org/abs/2408.02479>. Acesso em: 02 ago. 2026.

LEHMANN, Fabian et al. Software Engineering in the Era of LLMs. 2024. Disponível em: <https://arxiv.org/abs/2412.05229>. Acesso em: 02 ago. 2026.

MICROSOFT. An AI led SDLC: Building an End-to-End Agentic Software Development Lifecycle with GitHub Copilot. 2025. Disponível em: <https://learn.microsoft.com>. Acesso em: 02 ago. 2026.

MOHAGHEGHI, Milad et al. Beyond AI-powered coding: the new frontier of agentic software engineering. 2025. Disponível em: <https://arxiv.org/abs/2503.21353>. Acesso em: 02 ago. 2026.

QODO. AI-assisted code review. 2025. Disponível em: <https://www.qodo.ai>. Acesso em: 02 ago. 2026.

ROYCHOUDHURY, Abhik et al. Agentic Software Engineering: state and perspectives. 2025. Disponível em: <https://arxiv.org/abs/2505.02778>. Acesso em: 02 ago. 2026.

SOMMERVILLE, Ian. Engenharia de Software. 10. ed. São Paulo: Pearson, 2019.

YANG, John et al. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. 2024. Disponível em: <https://arxiv.org/abs/2405.15793>. Acesso em: 02 ago. 2026.

4 CONCLUSÃO

Este recorte investigativo confirmou a hipótese de que o SDLC AI-first constitui um novo contrato, e não uma automatização do ciclo clássico. A literatura analisada evidencia que a mudança de custo dominante — de horas-homem para tokens e contexto — reposiciona o artefato-mestre

do processo, que deixa de ser o documento para ser a especificação executável acompanhada de testes (LEHMANN et al., 2024; JIN et al., 2024). A capacidade demonstrada dos agentes em benchmarks como o SWE-bench reforça a viabilidade técnica da execução delegada (JIMENEZ et al., 2024; YANG et al., 2024).

O estudo também demonstrou que o papel humano não é eliminado, mas transformado. Na torre de controle do SDLC AI-first, o humano autoriza decolagens de fase, monitora o radar de observabilidade e arbitra desvios — funções de controle, não de execução (HODA, 2025; GRÖPLER et al., 2024). Essa redistribuição exige novas competências de especificação, leitura de evidência e decisão sob incerteza, com implicações diretas para a formação e para a governança (MOHAGHEGHI et al., 2025; ROYCHOUDHURY et al., 2025).

Para a prática, as recomendações são três: adotar a spec executável como contrato de todas as fases, delegar execução mantendo verificação adversarial independente, e tratar o custo de contexto como variável de projeto (MICROSOFT, 2025; ANTHROPIC, 2024). Para a pesquisa, ficam em aberto a mensuração de produtividade em ambiente agêntico e o estudo longitudinal dos efeitos da delegação sobre competências (FORSGREN, 2018; HODA, 2025). O recorte sugere que o próximo ciclo de pesquisa deve investigar o desenho dos contratos de delegação — o objeto de estudo da próxima obra desta série.

REFERÊNCIAS

ANTHROPIC. Building effective agents. 2025. Disponível em: <https://www.anthropic.com/research/building-effective-agents>. Acesso em: 02 ago. 2026.

ANTHROPIC. Introducing the Model Context Protocol. 2024b. Disponível em: <https://www.anthropic.com/news/model-context-protocol>. Acesso em: 02 ago. 2026.

WANG, Yuchen; GUO, Shangxin; TAN, Chee Wei. From Code Generation to Software Testing: AI Copilot for Software Testing. 2024. Disponível em: <https://arxiv.org/abs/2406.06574>. Acesso em: 02 ago. 2026.

CLARKE, Peter et al. Model Context Protocol: overview e adoção. 2025. Disponível em: <https://modelcontextprotocol.io>. Acesso em: 02 ago. 2026.

EVANS, Eric. Domain-Driven Design: Tackling Complexity in the Heart of Software. Boston: Addison-Wesley, 2003.

FORSGREN, Nicole; HUMBLE, Jez; KIM, Gene. Accelerate: The Science of Lean Software and DevOps. Portland: IT Revolution Press, 2018.

GOOGLE CLOUD. State of AI-assisted Software Development (DORA 2025). Disponível em: <https://dora.dev>. Acesso em: 02 ago. 2026.

GRÖPLER, Robin et al. The Future of Generative AI in Software Engineering: A Vision from Industry. 2024. Disponível em: <https://arxiv.org/abs/2411.17941>. Acesso em: 02 ago. 2026.

GURGUL, Vincent; GUBELA, Robin; LESSMANN, Stefan. The State of Generative AI in Software Development. 2024. Disponível em: <https://arxiv.org/abs/2410.18485>. Acesso em: 02 ago. 2026.

HINGEL, Paula. How AI Changes the SDLC: A Six-Stage Guide. Augment Code, 2026. Disponível em: <https://www.augmentcode.com>. Acesso em: 02 ago. 2026.

HODA, Rashina. Toward Agentic Software Engineering Beyond Code: Framing Vision, Values, and Vocabulary. 2025. Disponível em: <https://arxiv.org/abs/2505.10262>. Acesso em: 02 ago. 2026.

JIMENEZ, Carlos E. et al. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? ICLR, 2024. Disponível em: <https://arxiv.org/abs/2310.06770>. Acesso em: 02 ago. 2026.

JIN, Haolin et al. From LLMs to LLM-based Agents for Software Engineering: A Survey of Current, Challenges and Future. 2024. Disponível em: <https://arxiv.org/abs/2408.02479>. Acesso em: 02 ago. 2026.

LEHMANN, Fabian et al. Software Engineering in the Era of LLMs. 2024. Disponível em: <https://arxiv.org/abs/2412.05229>. Acesso em: 02 ago. 2026.

MICROSOFT. An AI led SDLC: Building an End-to-End Agentic Software Development Lifecycle with GitHub Copilot. 2025. Disponível em: <https://learn.microsoft.com>. Acesso em: 02 ago. 2026.

MOHAGHEGHI, Milad et al. Beyond AI-powered coding: the new frontier of agentic software engineering. 2025. Disponível em: <https://arxiv.org/abs/2503.21353>. Acesso em: 02 ago. 2026.

QODO. AI-assisted code review. 2025. Disponível em: <https://www.qodo.ai>. Acesso em: 02 ago. 2026.

ROYCHOUDHURY, Abhik et al. Agentic Software Engineering: state and perspectives. 2025. Disponível em: <https://arxiv.org/abs/2505.02778>. Acesso em: 02 ago. 2026.

SOMMERVILLE, Ian. Engenharia de Software. 10. ed. São Paulo: Pearson, 2019.

YANG, John et al. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. 2024. Disponível em: <https://arxiv.org/abs/2405.15793>. Acesso em: 02 ago. 2026.